Bases de datos

Tema 8

Davante

MySQL

Tipos de dato numéricos

Tipos de dato numéricos en MySQL, su ocupación en disco y valores:

INT (INTEGER): Ocupación de 4 bytes con valores entre -2147483648 y 2147483647 o entre 0 y 4294967295.

- **SMALLINT**: Ocupación de 2 bytes con valores entre -32768 y 32767 o entre 0 y 65535.
- **TINYINT**: Ocupación de 1 bytes con valores entre -128 y 127 o entre 0 y 255.
- **MEDIUMINT**: Ocupación de 3 bytes con valores entre -8388608 y 8388607 o entre 0 y 16777215.
- **BIGINT**: Ocupación de 8 bytes con valores entre -8388608 y 8388607 o entre 0 y 16777215.
- **DECIMAL (NUMERIC)**: Almacena los números de coma flotante como cadenas o string.
- **FLOAT (m,d)**: Almacena números de **coma flotante**, donde 'm' es el número de dígitos de la parte entera y 'd' el número de decimales.
- **DOUBLE (REAL)**: Almacena número de coma flotante con precisión doble. Igual que FLOAT, la diferencia es el rango de valores posibles.
- **BIT (BOOL, BOOLEAN)**: Número entero con valor 0 o 1.

MySQL

Tipos de dato con formato fecha

Listado de cada uno de los tipos de dato con formato fecha en MySQL, su ocupación en disco y valores:

- **DATE**: Válido para almacenar una fecha con año, mes y día, su rango oscila entre '1000-01-01' y '9999-12-31'.
- **DATETIME**: Almacena una fecha (año-mes-día) y una hora (horas-minutos-segundos), su rango oscila entre '1000-01-01 00:00:00' y '9999-12-31 23:59:59'.
- **TIME**: Válido para almacenar una hora (horas-minutos-segundos). Su rango de horas oscila entre -838-59-59 y 838-59-59. El formato almacenado es 'HH:MM:SS'.
- **TIMESTAMP**: Almacena una fecha y hora UTC. El rango de valores oscila entre '1970-01-01 00:00:01' y '2038-01-19 03:14:07'.
- **YEAR**: Almacena un año dado con 2 o 4 dígitos de longitud, por defecto son 4. El rango de valores oscila entre 1901 y 2155 con 4 dígitos. Mientras que con 2 dígitos el rango es desde 1970 a 2069 (70-69).

MySQL

Diferentes tipos de dato con formato string

Listado de cada uno de los tipos de dato con formato string en MySQL, su ocupación en disco y valores:

- **CHAR**: Ocupación fija cuya longitud comprende de 1 a 255 caracteres.
- **VARCHAR**: Ocupación variable cuya longitud comprende de 1 a 255 caracteres.
- **TINYBLOB**: Una longitud máxima de 255 caracteres. Válido para objetos binarios como son un fichero de texto, imágenes, ficheros de audio o vídeo. No distingue entre minúculas y mayúsculas.
- **BLOB**: Una longitud máxima de 65.535 caracteres. Válido para objetos binarios como son un fichero de texto, imágenes, ficheros de audio o vídeo. No distingue entre minúculas y mayúsculas.
- **MEDIUMBLOB**: Una longitud máxima de 16.777.215 caracteres. Válido para objetos binarios como son un fichero de texto, imágenes, ficheros de audio o vídeo. No distingue entre minúculas y mayúsculas.
- **LONGBLOB**: Una longitud máxima de 4.294.967.298 caracteres. Válido para objetos binarios como son un fichero de texto, imágenes, ficheros de audio o vídeo. No distingue entre minúculas y mayúsculas.
- **SET**: Almacena 0, uno o varios valores una lista con un máximo de 64 posibles valores.
- **ENUM**: Igual que **SET** pero solo puede almacenar un valor.

MySQL

- **TINYTEXT**: Una longitud máxima de 255 caracteres. Sirve para almecenar texto plano sin formato. Distingue entre minúculas y mayúsculas.
- **TEXT**:Una longitud máxima de 65.535 caracteres. Sirve para almecenar texto plano sin formato. Distingue entre minúculas y mayúsculas.
- **MEDIUMTEXT**:Una longitud máxima de 16.777.215 caracteres. Sirve para almecenar texto plano sin formato. Distingue entre minúculas y mayúsculas.
- **LONGTEXT**: Una longitud máxima de 4.294.967.298 caracteres. Sirve para almecenar texto plano sin formato. Distingue entre minúculas y mayúsculas.

MySQL

Crear una base de datos

CREATE DATABASE;

SHOW DATABASES;

USE prueba;

Desde el punto de vista de SQL, una base de datos es sólo un conjunto de relaciones (o tablas). A nivel de sistema operativo, cada base de datos se guarda en un directorio diferente.

Por tanto, crear una base de datos es una tarea muy simple. Claro que, en el momento de crearla, la base de datos estará vacía, es decir, no contendrá ninguna tabla.

Para crear una base de datos se usa una sentencia **CREATE DATABASE:**

```
mysql>CREATE DATABASE prueba;
Query OK, 1 row affected (0.03 sec)
```

MySQL

Para saber cuántas bases de datos existen en nuestro sistema usamos la sentencia **SHOW DATABASES:**

```
mysql>SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| mysql              |
| prueba             |
| test               |
+--------------------+
3 rows in set (0.00 sec)
```

MySQL

seleccionar una base de datos concreta se usa el comando USE que no es exactamente una sentencia SQL, sino más bien de una opción de MySQL. Esto nos permitirá obviar el nombre de la base de datos en consultas:

```
mysql>USE prueba;
Database changed
```

MySQL

Crear una tabla

CREATE TABLE ...
SHOW TABLES;

Veamos ahora la sentencia **CREATE TABLE** que sirve para crear tablas.
La sintaxis de esta sentencia es muy compleja, pero empezaremos con ejemplos sencillos.

En su forma más simple, la sentencia **CREATE TABLE** creará una tabla con las columnas que indiquemos. Crearemos, como ejemplo, una tabla que nos permitirá almacenar nombres de personas y sus fechas de nacimiento. Deberemos indicar el nombre de la tabla y los nombres y tipos de las columnas:

```
mysql> USE prueba
Database changed
mysql>CREATE TABLE gente (nombre VARCHAR(40), fecha DATE);
Query OK, 0 rows affected (0.53 sec)
```

Hemos creado una tabla llamada "gente" con dos columnas: "nombre" que puede contener cadenas de hasta 40 caracteres y "fecha" de tipo fecha.

MySQL

Podemos consultar cuántas tablas y qué nombres tienen en una base de datos, usando la sentencia **SHOW TABLES:**

```
mysql>SHOW TABLES;
+------------------+

| Tables_in_prueba |
+------------------+
| gente            |
+------------------+
1 row in set (0.01 sec)
```

MySQL

Además del tipo y el nombre, podemos definir valores por defecto, permitir o no que contengan valores nulos, crear una clave primaria, indexar…

La sintaxis para definir columnas de la tabla(atributos) es:

nombre_columna tipo [NOT NULL | NULL] [DEFAULT valor_por_defecto] [AUTO_INCREMENT] [[PRIMARY] KEY] [COMMENT 'string'] [definición_referencia]

Veamos cada una de las opciones por separado…

Valores nulos

… NOT NULL / NULL …

Al definir cada columna/atributo podemos decidir si podrá o no contener valores nulos.

Recordemos que aquellas columnas que son o forman parte de una clave primaria no pueden contener valores nulos.

Después veremos que si definimos una columna como clave primaria, automáticamente se impide que pueda contener valores nulos, pero este no es el único caso en que puede ser interesante impedir la asignación de valores nulos para una columna.

Cuando creamos una tabla la opción por defecto es que se permitan los valores nulos (***NULL***) y para que no se permitan, se especifica con ***NOT NULL***. Por ejemplo:

```
mysql>CREATE TABLE ciudad1 (nombre CHAR(20) NOT NULL, poblacion INT NULL);
Query OK, 0 rows affected (0.98 sec)
```

MySQL

Valores por defecto

... DEFAULT ...

Para cada columna/atributo también se puede definir, opcionalmente, un valor por defecto. El valor por defecto se asignará de forma automática a un atributo cuando no se especifique un valor determinado al añadir filas. Para ello usamos el comando DEFAULT dentro de la sentencia CREATE TABLE.

Si a un atributo se le declara que puede ser nulo y no se especifica un valor, por defecto, se usará NULL como valor por defecto. En el ejemplo anterior, el valor por defecto para *poblacion* es NULL.

Por ejemplo, si queremos que el valor por defecto para *poblacion* sea 5000, podemos crear la tabla como:

```
mysql>CREATE TABLE ciudad2 (nombre CHAR(20) NOT NULL, poblacion INT NULL
DEFAULT 5000);
Query OK, 0 rows affected (0.09 sec)
```

MySQL

Claves Primarias

... PRIMARY KEY...

Para *claves primarias* también se puede definir una clave primaria sobre un atributo, usando la palabra clave *KEY* o *PRIMARY KEY*.

Sólo puede existir una clave primaria en cada tabla, y el atributo sobre el que se define una clave primaria no puede tener valores *NULL*. Si esto no se especifica de forma explícita, **MySQL** lo hará de forma automática.
Por ejemplo, si queremos asignar la clave primaria a "*nombre*" de la tabla de ciudades, crearemos la tabla así:

```
mysql>CREATE TABLE ciudad3 (nombre CHAR(20) NOT NULL PRIMARY KEY, poblacion
INT NULL DEFAULT 5000);
Query OK, 0 rows affected (0.20 sec)
```

Usar **NOT NULL PRIMARY KEY** equivale a **PRIMARY KEY**, **NOT NULL KEY** o sencillamente **KEY**.

MySQL

Columnas Autoincrementadas

... AUTO_INCREMENT ...

Para *columnas autoincrementadas* en **MySQL** tenemos la posibilidad de crear una columna autoincrementada, aunque esta columna sólo puede ser de tipo entero.

Si al insertar una fila se omite el valor de la columna autoincrementada o si se inserta un valor nulo para esa columna, su valor se calcula automáticamente, tomando el valor más alto de esa columna y sumándole una unidad. Esto permite crear, de una forma sencilla, una columna con un valor único para cada fila de la tabla.

Generalmente, estas columnas se usan como claves primarias 'artificiales'. **MySQL** está optimizado para usar valores enteros como claves primarias, de modo que la combinación de clave primaria, que sea entera y autoincrementada es ideal para usarla como clave primaria artificial:

```
mysql>CREATE TABLE ciudad5 (clave INT AUTO_INCREMENT PRIMARY KEY,
-> nombre CHAR(20) NOT NULL,
-> poblacion INT NULL DEFAULT 5000);
Query OK, 0 rows affected (0.11 sec)
```

MySQL

Para establecer un número específico del que partiren el auto_increment hay que crear la tabla de la siguiente manera:

```
mysql> create table prueba (id int key auto_increment, name
varchar(20)) auto_increment=20;
Query OK, 0 rows affected (0.28 sec)

mysql> describe prueba;
+-------+-------------+------+-----+---------+----------------+
| Field | Type        | Null | Key | Default | Extra          |
+-------+-------------+------+-----+---------+----------------+
| id    | int(11)     | NO   | PRI | NULL    | auto_increment |
| name  | varchar(20) | YES  |     | NULL    |                |
+-------+-------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)
```

MySQL

Comentarios

... COMMENT...

Para los *comentarios*, adicionalmente, al crear la tabla, podemos añadir un comentario a cada columna. Este comentario sirve como información adicional sobre alguna característica especial de la columna, y entra en el apartado de documentación de la base de datos:

```
mysql>CREATE TABLE ciudad6
-> (clave INT AUTO_INCREMENT PRIMARY KEY COMMENT 'Clave principal',
-> nombre CHAR(50) NOT NULL,
-> poblacion INT NULL DEFAULT 5000);
Query OK, 0 rows affected (0.08 sec)
```

MySQL

borrar tabla

DROP TABLE ...

Para borrar una tabla usamos **DROP TABLE** de manera sencilla:

```
mysql> show tables;
+-----------------+
| Tables_in_mundo |
+-----------------+
| lista           |
| naciones        |
| prueba          |
| telef           |
+-----------------+
4 rows in set (0.00 sec)

mysql> DROP TABLE telef;
Query OK, 0 rows affected (0.32 sec)

mysql> show tables;
+-----------------+
| Tables_in_mundo |
+-----------------+
| lista           |
| naciones        |
| prueba          |
+-----------------+
3 rows in set (0.00 sec)
Query OK, 0 rows affected (0.08 sec)
```

MySQL

Modificar estructura de una tabla

ALTER TABLE ... RENAME...
ALTER TABLE ... DROP COLUMN...
ALTER TABLE ... DROP PRIMARY KEY...
ALTER TABLE ... ADD...
ALTER TABLE ... ADD...AFTER...
ALTER TABLE ... ADD...FIRST;
ALTER TABLE ... ADD...PRIMARY KEY...
ALTER TABLE ... MODIFY COLUMN...

Para modificar una tabla podemos usar el comando **ALTER TABLE:**
Por ejemplo, si queremos cambiar el nombre de la tabla:

```
mysql>ALTER TABLE ciudad6 RENAME granciudad;
Query OK, 0 rows affected (0.03 sec)
```

Para eliminar una columna de una tabla:

```
mysql>ALTER TABLE granciudad DROP COLUMN poblacion;
Query OK, 0 rows affected (1.28 sec)
Records: 0 Duplicates: 0  Warnings: 0
```

MySQL

Para eliminar la clave primaria de una columna:

```
mysql>ALTER TABLE granciudad DROP PRIMARY KEY;
Query OK, 1 row affected (1.20 sec)
Records: 1  Duplicates: 0  Warnings: 0
```

Para insertar una nueva columna al final de una tabla:

```
mysql>ALTER TABLE granciudad ADD fecha date;
Query OK, 1 row affected (1.20 sec)
Records: 1  Duplicates: 0  Warnings: 0
```

MySQL

Para añadir una nueva columna después de otra:

```
mysql>ALTER TABLE granciudad ADD origen VARCHAR(50) AFTER nombre;
Query OK, 0 rows affected (0.70 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Para añadir una nueva columna en la primera posición de la tabla:

```
mysql>ALTER TABLE granciudad ADD id INT  FIRST;
Query OK, 0 rows affected (0.72 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Asignar como clave primaria a una columna:

```
mysql>ALTER TABLE granciudad ADD PRIMARY KEY(id);
```

También podemos reestructurar una columna:

```
mysql>ALTER TABLE granciudad MODIFY COLUMN id INT auto_increment;
```

En este caso le hemos dotado a la columna id de la propiedad de autoincrementarse automáticamente.

MySQL

En este caso le hemos dotado a la columna id de la propiedad de autoincrementarse automáticamente. Cuando se añade una columna AUTO_INCREMENT, los valores de columna se llenan con una secuencia numérica automáticamente.

Se puede establecer el auto_increment usando ALTER TABLE de la siguiente manera:

```
mysql> alter table granciudad auto_increment = 40;
Query OK, 0 rows affected (0.08 sec)
```

O también con la opción **SET INSERT_ID**=value

```
mysql> set insert_ID=50;
Query OK, 0 rows affected (0.00 sec)
```

MySQL

Inserción de Filas

INSERT INTO... VALUES(...);
INSERT INTO... SET...

La forma más directa de insertar una fila nueva en una tabla es mediante una sentencia **INSERT**. En la forma más simple de esta sentencia debemos indicar la tabla a la que queremos añadir filas, y los valores de cada columna. Las columnas de tipo cadena o fechas deben estar entre comillas sencillas o dobles, para las columnas númericas esto no es imprescindible, aunque también pueden estar entrecomilladas.

```
mysql>INSERT INTO granciudad VALUES (1, 1, “barcelona”, “barcino”, “524-1-5”);
Query OK, 1 row affected (0.07 sec)
```

Introducir más de una fila a la vez:

```
mysql>INSERT INTO granciudad VALUES
→ (1, 1, “barcelona”, “barcino”, “524-1-5”),
→(2,2, “madrid”, “madrilus”, 1210-3-4”);
Query OK, 2 rows affected (0.09 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

MySQL

También es posible especificar la lista de columnas/atributos donde deseamos ingresar ciertos valores. La ventaja de esta forma es que no precisamos de conocer el orden exacto de los atributos en la tabla:

```
mysql>INSERT INTO granciudad (id, nombre) VALUES (4, "zaragoza");
Query OK, 1 row affected (0.13 sec)

mysql> SELECT * from granciudad;
+------+------+-----------+----------+------------+
| id2  | id   | nombre    | origen   | fecha      |
+------+------+-----------+----------+------------+
|    1 |    1 | barcelona | barcino  | 2011-01-01 |
|    2 |    2 | madrid    | madrilus | 1111-01-02 |
|    3 |    3 | valencia  | valentia | 3214-01-03 |
| NULL |    4 | zaragoza  | NULL     | NULL       |
+------+------+-----------+----------+------------+
4 rows in set (0.00 sec)
```

MySQL

Además existe una alternativa que consiste en indicar un valor para cada columna:

```
mysql>INSERT INTO granciudad SET
    -> id = 5,
    ->nombre = "Bilbao";
Query OK, 1 row affected (0.08 sec)

mysql> select * from granciudad;
+------+------+-----------+----------+------------+
| id2  | id   | nombre    | origen   | fecha      |
+------+------+-----------+----------+------------+
|    1 |    1 | barcelona | barcino  | 2011-01-01 |
|    2 |    2 | madrid    | madrilus | 1111-01-02 |
|    3 |    3 | valencia  | valentia | 3214-01-03 |
| NULL |    4 | zaragoza  | NULL     | NULL       |
| NULL |    5 | Bilbao    | NULL     | NULL       |
+------+------+-----------+----------+------------+
5 rows in set (0.01 sec)
```

MySQL

Eliminar Filas

DELETE FROM...
DELETE FROM ... WHERE...
DELETE FROM ... WHERE...ORDER BY ... ASC/DESC;
DELETE FROM ... ORDER BY ... ASC/DESC... LIMIT...;

Con la sentencia **DELETE** podemos eliminar filas de la tabla:

```
mysql>DELETE FROM ciudad3;
Query OK, 5 rows affected (0.05 sec)
```

En este caso se han eliminado TODAS las filas de la tabla 'ciudades'.